



Syntax & Disciplined Coding Style

Criterion 7 · Phase 3: Core JavaScript Coding · 6 Marks

SkillsTrack Learner Support Portal — JavaScript Programmer (NSSA TPC)

No single deliverable — this is assessed throughout the whole codebase.



```
$ eslint .
```

```
✓ 0 problems
```

```
clean, consistent, readable
```

What This Criterion Wants

Code that reads like it was written on purpose.



Consistent formatting

Indentation, spacing and style stay the same across every file.



Meaningful names

Variables and functions say what they hold or do — no x, temp, or data2.



Comments where needed

Explain the why behind tricky logic, not the obvious what.



Sensible files & folders

Code organised into logical files, not one giant script.



Small Weight, Big Ripple

Only 6 marks are attached directly — but this criterion colours how every other code-based answer is read.

Every other code-based criterion is read through this lens

Markers form a first impression from formatting alone

Messy code makes markers doubt correct answers too

“Messy code makes markers doubt your other answers too.”

Consistent Formatting

Same Rules, Every File



One style, applied everywhere — automatically.

- Same indentation (2 or 4 spaces) across every file, no mixing
- Consistent spacing around operators, braces and commas
- Prettier + ESLint from Criterion 5 enforce this automatically
- Run `npm run format` and `npm run lint` before every commit

.PRETTIERRC — THE RULES

```
.prettierrc > ...
1 {
2   "singleQuote": false,
3   "semi": true,
4   "tabWidth": 2,
5   "printWidth": 80,
6   "trailingComma": "es5"
7 }
8
```

NPX PRETTIER — CHECK, THEN WRITE

```
PS C:\Users\0809285423086\Desktop\Javascript_Program_2026> npx prettier --check .
Checking formatting...
[warn] .github/workflows/lint.yml
[warn] .vscode/settings.json
[warn] CHANGELOG.md
[warn] eslint.config.mjs
[warn] package-lock.json
[warn] package.json
[warn] README.md
[warn] Code style issues found in 7 files. Run Prettier with --write to fix.
PS C:\Users\0809285423086\Desktop\Javascript_Program_2026> npx prettier --write .
.github/workflows/lint.yml 68ms
.prettierrc 58ms (unchanged)
.vscode/settings.json 3ms
CHANGELOG.md 32ms
designs/Figma Link.md 4ms (unchanged)
eslint.config.mjs 18ms
package-lock.json 50ms
package.json 7ms
README.md 97ms
```

`--check` flags 7 files with style issues; `--write` fixes every one automatically.

Meaningful Names

Say What It Holds or Does



A name should explain itself — no digging through the file to understand it.

- Variables describe the value they hold: `currentDay`, `currentTime`, not `d` or `t`
- Functions describe the action they perform: `selectDay()`, `selectType()`
- `eslint.config.mjs` enforces it: `camelcase`, `no-var` and `prefer-const` are errors
- The linter actually catches slips — `var currentDay` flagged as `no-var`

ESLINT.CONFIG.MJS — THE RULE

```
1 import js from "@eslint/js";
2
3 export default [
4   js.configs.recommended,
5   {
6     rules: {
7       "no-var": "error",
8       "prefer-const": "error",
9       "eqeqeq": "error",
10      "no-unused-vars": "warn",
11      camelcase: ["error", { properties: "always" }],
12    },
13  ],
14 ];
```

CAUGHT ON REAL CODE

```
1 // booking.js
2 // Handles session-type, day, and time selection, plus the request
3 // summary and submission, for the "Book a Support Session" page.
4
5 var currentDay = "";
6 let currentTime = "";
7
8 /**
9  * Displays the chosen session type in the request summary.
10  * @param {string} value - Label of the selected session type
11  * (e.g. "Progress Review").
12  */
13 function selectType(value) {
14   document.getElementById("summaryType").innerHTML = value;
15 }
16
17 /**
18  * Records the selected day and refreshes the combined day/time summary.
19  * @param {string} value - The selected day (e.g. "Mon 11").
20  */
21 function selectDay(value) {
22   currentDay = value;
23   updatedDayTimeSummary();
24 }
```

PROBLEMS

- ✖ 1 problem (1 error, 0 warnings)
1 error and 0 warnings potentially fixable with the '--fix' option.

✖ 1 error Unexpected var, use let or const instead no-var

Comments Where Needed

Explain the Why, Not the What



A comment earns its place by saving the next reader time.

- JSDoc-style blocks on shared functions: what it does, @param, return
- Inline notes only where logic isn't obvious from the code itself
- Skip comments that just restate the line below them
- A short file header explaining what the file is responsible for

REAL FILE — booking.js

```
1 // booking.js
2 // Handles session-type, day, and time selection, plus the request
3 // summary and submission, for the "Book a Support Session" page.
4
5 var currentTime = "";
6 let currentDay = "";
7
8 /**
9  * Displays the chosen session type in the request summary.
10  * @param {string} value - Label of the selected session type
11  *   (e.g. "Progress Review").
12  */
13 function selectType(value) {
14   document.getElementById("summaryType").innerHTML = value;
15 }
16
17 /**
18  * Records the selected day and refreshes the combined day/time summary.
19  * @param {string} value - The selected day (e.g. "Mon 11").
20  */
21 function selectDay(value) {
22   currentDay = value;
23   updateDayTimeSummary();
24 }
25
26 /**
27  * Records the selected time and refreshes the combined day/time summary.
28  * @param {string} value - The selected time slot (e.g. "9:00 AM").
29  */
30 function selectTime(value) {
31   currentTime = value;
32   updateDayTimeSummary();
33 }
34
35 /**
36  * Builds and displays the "day - time" summary text.
37  * Shared by selectDay and selectTime so the string-building logic
38  * only lives in one place.
39  */
```

Three JSDoc blocks in a row — `selectType`, `selectDay`, `selectTime` — each with a purpose and @param.

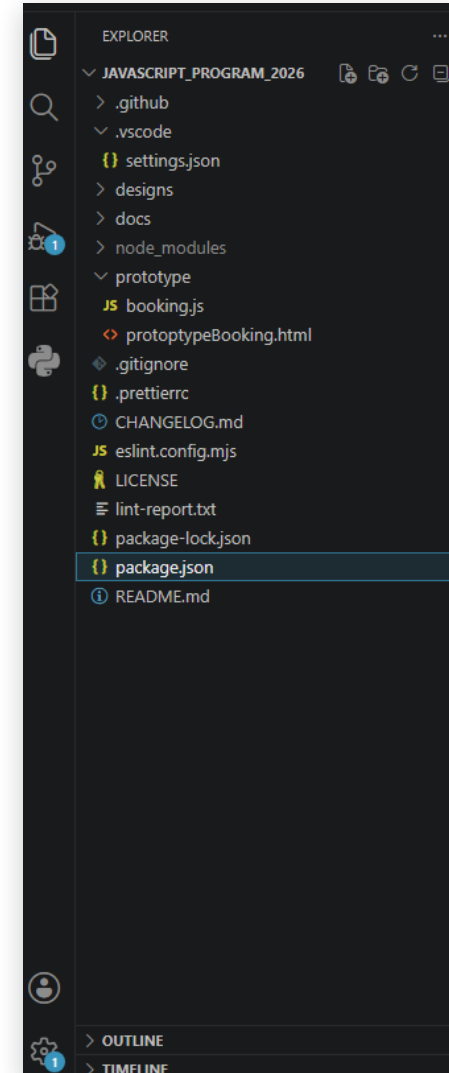
Sensible Files & Folders

Organised, Not Dumped



Every file has an obvious reason to exist where it does.

- Top-level folders separate concerns: docs, designs, prototype
- Config lives at the root: .eslintrc, .prettierrc, package.json
- One feature's logic stays in one file — booking.js owns booking
- No stray scripts loose in the project root



How This Gets Assessed

No Single Deliverable



There's nothing to upload for this one — the whole codebase is the evidence.

- Run `npm run lint` and `npm run format` before every submission
- Markers read every file, not one screenshot — consistency has to hold everywhere
- The ESLint + Prettier setup from Criterion 5 does the enforcing automatically
- A lint report can be piped straight to a file for a written record

`$ npm run lint > lint-report.txt`

```
0805085433086@RDS MINGW64 ~/Desktop/Javascript_Program_2026 (1wandle)
$ npm run lint > lint-report.txt
npm error
npm error Did you mean this?
npm error   npm link # Symlink a package folder
npm error
npm error To see a list of scripts, run:
npm error   npm run
npm error A complete log of this run can be found in: C:\Users\0805085433086\AppData\Local\npm-cache\_logs\2026-08-12T11_09_16_422Z-debug-0.log
0805085433086@RDS MINGW64 ~/Desktop/Javascript_Program_2026 (1wandle)
```

Real terminal output — the lint script piped to a saved report file.

Criterion 7 — Covered

Discipline that shows up in every file, not one screenshot.

- ✓ Formatting stays consistent — enforced by Prettier + ESLint
- ✓ Variables and functions named for what they hold or do
- ✓ Comments explain the why, placed only where they earn it
- ✓ Files and folders organised by feature, not dumped flat
- ✓ Linter and formatter run clean before every submission

6

out of 6 marks

"Messy code makes markers doubt your other answers too."